

Helix OTA — CLAUDE.md

INHERITED FROM constitution/CLAUDE.md

All rules in constitution/CLAUDE.md and the constitution/Constitution.md it references apply unconditionally to this project. Project-specific rules below extend them — they do NOT weaken or override any universal clause.

When this file disagrees with the constitution submodule, the constitution wins.

@constitution/CLAUDE.md

Project overview

Helix OTA is an enterprise-grade over-the-air update system: a custom Go control plane (Gin modular monolith) driving native Android A/B updates (update_engine + AVB/dm-verity + auto-rollback) for RK3588 / Orange Pi 5 Max targets, with a roadmap to Linux, Windows, and other operating systems. Reusable building bricks live in submodules/ (six ota-* modules) and the dev/runtime infrastructure in containers/.

Project-specific MANDATORY constraints

Build / packaging

- Server: Go module `github.com/HelixDevelopment/helix_ota/server` (server/). `go build ./...`, `gofmt`, `go vet` MUST be clean.
- Submodule Go bricks build + test from their own module roots under `submodules/`.
- Android bricks (ota-update-engine-bridge, ota-android-agent) build with AGP 8.5.2 + plain `kotlin.android` on Gradle 9.5 (Kotlin MPP does NOT build on Gradle 9.5 — see CONTINUATION handoff).

- Documentation source of truth is Markdown + Mermaid; PDF/HTML/DOCX and draw.io/SVG/UML/PNG are generated artifacts (scripts/export_docs.sh).

Test / verification

- Server: `cd server && go test ./...` (httptest integration suite).
- Inheritance gate (this wiring): `bash tests/inheritance_gate.sh`.
- Full inheritance + paired-mutation proof: `bash tests/test_constitution_inheritance.sh`.

Deployment / flash / publish

- Multi-upstream: the parent and every owned submodule push to all four upstreams (GitHub primary + GitLab + GitFlic + GitVerse). The parent's origin remote fans out pushes to all four.

Feature video recording + Status doc mandates (§11.4.153–§11.4.159)

- docs/features/Status.md + Status_Summary.md MUST always reflect every feature with per-row video-recording confirmation (HTML+PDF+DOCX exports in sync via docs_chain). Code-present features missing from the table are §11.4.153 violations.
- All recordings MUST be window-scoped (not whole-desktop) per §11.4.154. A new recording run removes its own prior in-scope recordings first.
- All recording filenames MUST start with <PREFIX>--- per §11.4.155 (triple-hyphen separator, canonical form <PREFIX>---<feature-or-scope>---<run-id>.<ext>). PREFIX is resolved from HELIX_RELEASE_PREFIX env var or the lowercased project-root dir name.
- **Recording path override:** All recordings use \$HOME/Downloads as the default save path per §11.4.158(D). This project does NOT override the default — no separate recording-path variable is defined. Recordings at non-\$HOME/Downloads paths (e.g. /Volumes/T7/Downloads/Recordings/) are a temporary convenience and MUST be migrated before release tagging.
- All CI/CD automation is DISABLED per §11.4.156. No active .yaml workflow files exist; enforcement is local-only via pre_build_verification.sh.
- GEMINI.md lockstep per §11.4.157 — all per-agent context carriers carry the same highest §11.4.N anchor.

- Every recording's on-screen content MUST be machine-read and verified as a genuine working result per §11.4.158. A video without read-the-screen verification is not evidence.
- §11.4.159 — window-specific MP4 with vision validation, expected-content specification before recording, content-verification workflow
(SPECIFY → RECORD → EXTRACT → VERIFY → CHECK → ACCEPT), terminal cleanup per window id, and root cause analysis on rejected recordings.

Universal mandates propagation (§11.4.160–§11.4.166)

- §11.4.160 — every feature/QA recording is processed through a vision/OCR bridge that reads on-screen content and confirms expected results BEFORE acceptance (extends §11.4.158/§11.4.159).
- §11.4.161 — all containerized workloads (the containers/ dev/ runtime infra) use Podman in rootless mode via the vasic-digital/ containers submodule (§11.4.76); Docker-rootful / sudo / root escalation is forbidden.
- §11.4.162 — any user-facing interface MUST use the OpenDesign design-token system (light+dark themes, visual-regression covered); latent until this project ships a UI surface.
- §11.4.163 — every recorded artifact passes the MEDIA VALIDATION pipeline (OCR / transcription / text-parse vs SPECIFY-phase patterns, self-validated golden-good/golden-bad analyzer) before acceptance.
- §11.4.164 — after every constitution pull, run constitution/ scripts/post_update_hook.sh (inherited by reference, NEVER copied) to detect / register / install changed skills / MCP / hooks / scripts.
- §11.4.165 — every code change OR recorded media artifact passes an INDEPENDENT verifier (structurally separate from the author) that iterates to a zero-finding GO.
- §11.4.166 — Semgrep static analysis is MANDATORY: installed + on PATH for all users, scanned (semgrep scan --config auto --error) before every commit / push, blocking on findings. Scripts inherited by reference from constitution/scripts/semgrep/* (NEVER copied); MCP wired per §11.4.78 step 3. Scope: server/, submodules/, scripts/.

Universal mandates propagation (§11.4.167–§11.4.186 + §12.12)

- §11.4.167 — every BIG feature/large fix MUST develop as its own isolated feature work-stream (CoW/reflink project copy, own feature/<slug> branch
 - tags, own builds), kept separate from trunk until operator-approved after full retest; trunk merged in regularly; submodule branch/tag cascade; single-builder + per-device exclusive test queues (composes §11.4.58/.103/.113/.142/.145). Gate CM-COVENANT-114-167-PROPAGATION.
- §11.4.168 — every exported document (HTML/PDF/DOCX) MUST pass independent validation (reviewer structurally separate from the generator) across CONTENT, TEXTUAL, and FULL-VISUAL layers, verified by rendering + OCR with a self-validated golden-good/golden-bad analyzer (composes §11.4.65/.107/.117/.134/.142). Gate CM-COVENANT-114-168-PROPAGATION.
- §11.4.169 — every project MUST cover the closed enumerated test-type set (unit/integration/e2e/full-automation/Challenges/HelixQA/DDoS/security/ stress+chaos/concurrency/race-deadlock/memory/benchmarking), each PASS citing rock-solid captured physical evidence (composes §11.4.25/.27/.50/.52/.69/.85/.107). Gate CM-COVENANT-114-169-PROPAGATION.
- §11.4.170 — every UI-surface change MUST be proven by device-independent host-rendered pixels (Compose/Playwright/Storybook/snapshot-testing class) per screen×state×{light,dark} theme, validated by golden image-diff AND an OCR/vision layout oracle; value/token-equality unit tests are FORBIDDEN as the sole proof (composes §11.4.107/.153/.158/.159/.160/.162/.168). Gate CM-COVENANT-114-170-PROPAGATION.
- §11.4.171 — every workable item MUST carry a ≥5-7-sentence plain-language description (what/why/how/who-benefits/expected-outcome) understandable by non-developers, the SQLite DB description column the single source of truth mirrored into every derived doc (composes §11.4.15/.16/.91/.93/.148). Gate CM-COVENANT-114-171-PROPAGATION.
- §11.4.172 — every project MUST maintain a living production-readiness planning document with realistic timeline projections from measured velocity, danger-zone/risk identification, and critical-path analysis, updated monthly or on ≥10% item-count change (composes §11.4.6/.40/.42/.93/.108). Gate CM-COVENANT-114-172-PROPAGATION.
- §11.4.173 — every build of every component MUST run inside a specialized build container via the containers submodule,

- distributed to the designated remote build host (never the bare host), artifacts brought back for use/flashing (composes §11.4.28/.74/.76/.108/.161). Gate CM-COVENANT-114-173-PROPAGATION.
- §11.4.174 — before inspecting or acting on any process/build/daemon/port/ lock on a shared host, positively verify the target is OURS (cwd/argv/ recorded-PID/lock-path) — never a loose name-match pgrep; never kill a process not positively ours (composes §11.4.66/.101/.133/.147). Gate CM-COVENANT-114-174-PROPAGATION.
 - §11.4.176 — conflict-free multi-track work-division: (A) exactly-once work-item/logical-group claim registry; (B) capability-aware deadlock-proof device-lock (all-or-nothing, non-blocking, TTL-reap breaking Coffman conditions); (C) universal/decoupled + evidence-based resource tuning (composes §11.4.58/.116/.119/.147/.167). Gate CM-COVENANT-114-176-PROPAGATION.
 - §11.4.177 — no project-specific script/hook/alias may be wired into a global/shared developer-tooling PATH; shared tooling MUST be project-agnostic and operate on the invocation directory, never a hardcoded project path (composes §11.4.28/.29/.35). Gate CM-COVENANT-114-177-PROPAGATION.
 - §11.4.178 — parallel work streams sharing hardware/git/session/ lock namespaces MUST be addressed by a track-qualified identity (<project>__<track>__<role>), never a bare basename — prevents session/lock/log/device cross-wiring (composes §11.4.111/.119/.176). Gate CM-COVENANT-114-178-PROPAGATION.
 - §11.4.179 — parallel git work streams requiring corruption-isolation MUST each be an independent repo with its OWN .git (own object store/index/lock namespace), NOT git worktree checkouts sharing one common .git (composes §11.4.58/.119/.167/ §9.2). Gate CM-COVENANT-114-179-PROPAGATION.
 - §11.4.180 — every commit/push wrapper MUST auto-reap a git lock whose recorded holder PID is dead (or, absent a PID, stale past a defined threshold with no live holder) before acquiring its own lock — NEVER remove a lock whose holder is alive (composes §11.4.84/.88/ §9.2/.179). Gate CM-COVENANT-114-180-PROPAGATION.
 - §11.4.181 — one feature/logical group of workable items maps to EXACTLY ONE canonical branch name used identically on the main repo AND every touched owned submodule, recorded once in the claim registry, never re-invented (composes §11.4.29/.93/.113/.167/.176). Gate CM-COVENANT-114-181-PROPAGATION.
 - §11.4.182 — every agent/subagent label and operator-facing work-stream reference MUST start with a (T<N>/<branch> - <alias>) prefix, deterministically derived (never guessed), mechanically enforced by a PreToolUse guard hook (composes §11.4.75/.109/.178). Gate CM-COVENANT-114-182-PROPAGATION.

- §11.4.183 — every track MUST maximize applicable multi-agent working approaches (subagent-driven dev, independent review agents, parallel background streams) and apply the ENTIRE constitution — nothing skipped — with zero false/faulty/unverified results anywhere (composes §11.4.20/.58/.70/.125/.142/.165). Gate CM-COVENANT-114-183-PROPAGATION.
- §11.4.184 — the SonarQube scanner CLI (sonar-scanner) MUST be installed AND durably PATH-discoverable via shell rc, plus the shared constitution/scripts/sonarqube/ tooling consumed by reference (never copied); proven GREEN by install-check exit 0, never assumed; rootless podman only (composes §11.4.28/.74/.161/.166). Gate CM-COVENANT-114-184-PROPAGATION.
- §11.4.185 — no scope of work / release may be considered fully completed until it has received MANUAL testing confirmation by the project's QA team as the FINAL step — every automated gate is necessary but NOT sufficient; the agent hands off and waits, never self-certifies the manual step (composes §11.4.40/.52/.108/.126). Gate CM-COVENANT-114-185-PROPAGATION.
- §11.4.186 — any project maintaining more than one representation of the same tracked data MUST enforce cross-document consistency as a deterministic PASS/FAIL/SKIP gate running BEFORE any export render, doc/DB sync verify, or doc-set commit — never an after-the-fact audit; five decidable check families + self-validated analyzer (composes §11.4.86/.93/.106/.107/.148). Gate CM-COVENANT-114-186-PROPAGATION.
- §12.12 — heavy parallel subagent/multi-process work is bounded by the OS per-user process/thread limit (ulimit -u / RLIMIT_NPROC); check thread headroom before scaling parallelism and treat exhaustion as a §12 host-safety event that yields unconditionally — an orthogonal axis to the §12.6 memory ceiling (composes §12.6/.7/ §11.4.58/.101/.103/.122). Gate CM-COVENANT-12-12-PROPAGATION.

Project-specific architecture notes

- Persistence seam: `server/internal/store.Repository`. MVP wires the in-memory implementation; the `pgx/PostgreSQL` implementation is the production target (architecture.md §4).
 - Trust boundary (security): the artifact-signature verification key comes ONLY from server configuration — never from the request (`server/internal/api/handlers_artifact.go:resolvePublicKey`).
-

Project overrides of universal rules

(none — this project does not override any universal clause)